

实验作业 L3：攻击实验 (The Attack Lab)

3132102410 2018 年秋季

发布日期：12 月 6 日

截止日期：12 月 20 日星期五

本次作业涉及对两个具有不同安全漏洞的程序总共发起五次攻击。通过本实验您将获得以下成果：

- 您将学习到攻击者在程序未充分防范缓冲区溢出时，可以利用各种安全漏洞的不同方式。
- 通过这一点，您将更好地理解如何编写更安全的程序，以及编译器和操作系统为降低程序漏洞提供的某些特性。
- 您将对 x86-64 机器码的栈和参数传递机制获得更深入的理解。
- 您将对 x86-64 指令的编码方式获得更深入的理解。
- 您将获得使用 GDB 和 OBJDUMP 等调试工具的更多经验。

注意：在本实验中，您将亲身体验用于利用操作系统和网络服务器安全漏洞的方法。我们的目的是帮助您了解程序的运行时操作，并理解这些安全漏洞的性质，以便您在编写系统代码时能够避免它们。我们不纵容使用任何其他形式的攻击来获取对任何系统资源的未经授权的访问。您需要学习 CS:APP3e 教科书中的第 3.10.3 节和 3.10.4 节作为本实验的参考资料。

2 准备工作

与往常一样，这是一个个人项目。您将为您量身定制的目标程序生成攻击。

2.1 获取文件

您的主目录中有一个 `targetk.tar` 文件，其中 `k` 是您目标程序的唯一编号。然后运行以下命令：`tar -xvf targetk.tar`。这将解压出一个名为 `targetk` 的目录，其中包含以下文件：

- `README.txt`：描述目录内容的文档。
- `ctarget`：一个易受**代码注入 (Code-Injection)** 攻击的可执行程序。
- `rtarget`：一个易受**返回导向编程 (Return-Oriented-Programming)** 攻击的可执行程序。
- `cookie.txt`：一个 8 位十六进制代码，您将用作攻击中的唯一标识符。
- `farm.c`：目标程序的“gadget farm”源代码，您将在生成返回导向编程攻击时使用。
- `hex2raw`：一个用于生成攻击字符串的实用程序。

在以下说明中，我们假设您已将文件复制到受保护的本地目录中，并且正在该本地目录中执行程序。

2.2 重要事项

以下是关于本实验有效解决方案的一些重要规则总结。当您第一次阅读本文档时，这些要点可能不太容易理解。它们在此处作为您开始工作后的核心规则参考。

- 您必须在与生成您的目标程序相似的机器上进行作业。
- 您的解决方案不得使用攻击来规避程序中的验证代码。具体而言，您在攻击字符串中嵌入的、供 `ret` 指令使用的任何地址都应指向以下目的地之一：

- 函数 `touch1`、`touch2` 或 `touch3` 的地址。
- 您注入的代码的地址。
- 来自 gadget farm 的某个 gadget 的地址。
- 您只能从 `rtarget` 文件中，利用函数 `start_farm` 和 `end_farm` 之间的地址范围来构造 gadget。

3 目标程序

`CTARGET` 和 `RTARGET` 都从标准输入读取字符串。它们通过下面定义的函数 `getbuf` 来实现：

```
1 unsigned getbuf()
2 {
3     char buf[BUFFER_SIZE];
4     Gets(buf);
5     return 1;
6 }
```

函数 `Gets` 类似于标准库函数 `gets`——它从标准输入读取一个字符串（以 `\n` 或文件结束符终止），并将其（连同空终止符）存储在指定的目的地。在这段代码中，您可以看到目的地是一个数组 `buf`，声明为具有 `BUFFER_SIZE` 字节。在生成您的目标程序时，`BUFFER_SIZE` 是一个特定于您的程序版本的编译时常量。

函数 `Gets()` 和 `gets()` 无法确定它们的目标缓冲区是否足够大以存储它们读取的字符串。它们只是简单地复制字节序列，可能会超出为目的地分配的存储空间的边界。

如果用户键入并由 `getbuf` 读取的字符串足够短，很明显 `getbuf` 将返回 1，如下面的执行示例所示：

```
1 unix> ./ctarget
2 Cookie: 0x1a7dd803
3 Type string: Keep it short!
4 No exploit. Getbuf returned 0x1
5 Normal return
```

如果您键入一个长字符串，通常会发生错误：

```
1 unix> ./ctarget
2 Cookie: 0x1a7dd803
3 Type string: This is not a very interesting string, but it has the property ...
4 Ouch!: You caused a segmentation fault!
5 Better luck next time
```

（请注意，显示的 cookie 值将与您的不同。）程序 `RTARGET` 将具有相同的行为。正如错误消息所示，缓冲区溢出通常会导致程序状态损坏，从而导致内存访问错误。您的任务是用更巧妙的字符串来馈送 `CTARGET` 和 `RTARGET`，使它们执行更有趣的操作。这些被称为**exploit strings**（漏洞利用字符串）。

`CTARGET` 和 `RTARGET` 都接受几个不同的命令行参数：

- `-h`：打印可能的命令行参数列表
- `-q`：不向评分服务器发送结果
- `-i FILE`：从文件而不是标准输入提供输入

您的 exploit string 通常会包含不对应于可打印字符 ASCII 值的字节值。程序 `HEX2RAW` 将使您能够生成这些原始字符串。有关如何使用 `HEX2RAW` 的更多信息，请参阅附录 A。

重要事项：

- 您的 exploit string 在任何中间位置都**不得**包含字节值 `0x0a`，因为这是换行符 (`\n`) 的 ASCII 码。当 `gets` 遇到此字节时，它会假定您打算终止字符串。
- `HEX2RAW` 期望两个十六进制数字的字节值，并用一个或多个空格分隔。因此，如果您想创建一个十六进制值为 0 的字节，则需要将其写为 `00`。要创建字 `0xdeadbeef`，您应该将 `"ef be ad de"` 传递给 `HEX2RAW`（注意小端字节序所需的反转）。

当您正确解决其中一个级别时，您的目标程序将自动向评分服务器发送通知。例如：

```
1 | unix> ./hex2raw <ctarget.12.txt | ./ctarget
2 | Cookie: 0x1a7dd803
3 | Type string: Touch2!: You called touch2(0x1a7dd803)
4 | valid solution for level 2 with target ctarget
5 | PASSED: Sent exploit string to server to be validated.
6 | NICE JOB!
```

服务器将测试您的 exploit string 以确保它确实有效，并将更新 Attacklab 计分板页面，表明您的用户 ID 已完成该阶段。

您可以通过将 Web 浏览器指向 `http://10.112.152.199:19320/scoreboard` 来查看计分板。

与 Bomb Lab 不同，本实验中犯错没有惩罚。请随意使用您喜欢的任何字符串向 `CTARGET` 和 `RTARGET` 发起攻击。

阶段	程序	级别	方法	函数	分数
1	CTARGET	1	CI	touch1	10
2	CTARGET	2	CI	touch2	25
3	CTARGET	3	CI	touch3	25
4	RTARGET	2	ROP	touch2	35
5	RTARGET	3	ROP	touch3	5
CI:	代码注入				
ROP:	返回导向编程				

图 1：攻击实验阶段总结

图 1 总结了实验的五个阶段。可以看出，前三个涉及对 `CTARGET` 的**代码注入 (CI)** 攻击，而后两个涉及对 `RTARGET` 的**返回导向编程 (ROP)** 攻击。

4 第一部分：代码注入攻击 (CI)

对于前三个阶段，您的 exploit string 将攻击 `CTARGET`。此程序的设置方式使得堆栈位置在每次运行之间保持一致，并且堆栈上的数据可以被视为可执行代码。这些特性使得程序容易受到 exploit string 包含可执行代码字节编码的攻击。

4.1 级别 1（重定向）

对于阶段 1，您不会注入新代码。相反，您的 exploit string 将重定向程序以执行一个现有的过程。

函数 `getbuf` 是在 `CTARGET` 中由一个名为 `test` 的函数调用的，该函数具有以下 C 代码：

```
1 void test()
2 {
3     int val;
4     val = getbuf();
5     printf("No exploit. Getbuf returned 0x%x\n", val);
6 }
```

当 `getbuf` 执行其 `return` 语句（`getbuf` 的第 5 行）时，程序通常会恢复在函数 `test` 中的执行（在该函数的第 5 行）。我们希望改变这种行为。在 `ctarget` 文件中，有一个函数 `touch1` 的代码，具有以下 C 表示：

```
1 void touch1()
2 {
3     vlevel = 1; /* Part of validation protocol */
4     printf("Touch1!: You called touch1()\n");
5     validate(1);
6     exit(0);
7 }
```

您的任务是让 `CTARGET` 在 `getbuf` 执行其 `return` 语句时执行 `touch1` 的代码，而不是返回到 `test`。请注意，您的 exploit string 也可能破坏与此阶段不直接相关的堆栈部分，但这不会造成问题，因为 `touch1` 会直接导致程序退出。

一些建议：

- 您为该级别设计 exploit string 所需的所有信息都可以通过检查 `CTARGET` 的反汇编版本来确定。使用 `objdump -d` 来获取此反汇编版本。
- 关键思想是定位 `touch1` 起始地址的字节表示，使得 `getbuf` 代码末尾的 `ret` 指令将控制权转移到 `touch1`。
- 注意字节顺序。
- 您可能希望使用 GDB 逐步执行 `getbuf` 的最后几条指令，以确保它正在执行正确的操作。
- `buf` 在 `getbuf` 栈帧中的位置取决于编译时常量 `BUFFER_SIZE` 的值以及 GCC 使用的分配策略。您需要检查反汇编代码以确定其位置。

4.2 级别 2（指定参数）

阶段 2 涉及注入少量代码作为您的 exploit string 的一部分。

在 `ctarget` 文件中，有一个函数 `touch2` 的代码，具有以下 C 表示：

```

1 1 void touch2(unsigned val)
2 2{
3 3 vlevel = 2; /* Part of validation protocol */
4 4 if (val == cookie) {
5 5 printf("Touch2!: You called touch2(0x%.8x)\n", val);
6 6 validate(2);
7 7} else {
8 8 printf("Misfire: You called touch2(0x%.8x)\n", val);
9 9 fail(2);
10 10}
11 11 exit(0);
12 12}

```

您的任务是让 `CTARGET` 执行 `touch2` 的代码，而不是返回到 `test`。但是，在这种情况下，您必须让 `touch2` 看起来像是您已将您的 **cookie** 作为其参数传递。

一些建议：

- 您需要以某种方式定位您注入代码地址的字节表示，使得 `getbuf` 代码末尾的 `ret` 指令将控制权转移到它。
- 回想一下，函数的第一个参数是通过寄存器 `%rdi` 传递的。
- 您注入的代码应该将该寄存器设置为您的 **cookie**，然后使用 `ret` 指令将控制权转移到 `touch2` 中的第一条指令。
- 不要尝试在您的 exploit code 中使用 `jmp` 或 `call` 指令。这些指令的目标地址编码很难制定。对所有控制转移使用 `ret` 指令，即使您不是从调用返回。
- 请参阅附录 B 中关于如何使用工具生成指令序列的字节级表示的讨论。

4.3 级别 3（栈中覆盖指定信息）

阶段 3 也涉及代码注入攻击，但需要传递一个字符串作为参数。

在 `ctarget` 文件中，有函数 `hexmatch` 和 `touch3` 的代码，具有以下 C 表示：

```

1 1 /* Compare string to hex representation of unsigned value */
2 2 int hexmatch(unsigned val, char *sval)
3 3{
4 4 char cbuf[110];
5 5 /* Make position of check string unpredictable */
6 6 char *s = cbuf + random() % 100;
7 7 sprintf(s, "%.8x", val);
8 8 return strncmp(sval, s, 9) == 0;
9 9}
10 10
11 11 void touch3(char *sval)
12 12 {
13 13 vlevel = 3; /* Part of validation protocol */
14 14 if (hexmatch(cookie, sval)) {
15 15 printf("Touch3!: You called touch3(\"%s\")\n", sval);
16 16 validate(3);
17 17} else {
18 18 printf("Misfire: You called touch3(\"%s\")\n", sval);
19 19 fail(3);

```

```
20 20}
21 21 exit(0);
22 22}
```

您的任务是让 `CTARGET` 执行 `touch3` 的代码，而不是返回到 `test`。您必须让 `touch3` 看起来像是您已将您的 **cookie** 的字符串表示作为其参数传递。

一些建议：

- 您需要在 exploit string 中包含您的 **cookie** 的字符串表示。该字符串应包含八位十六进制数字（从最高位到最低位排列），**没有**前导的 `"0x"`。
- 回想一下，字符串在 C 中表示为一系列字节，后跟一个值为 0 的字节。在任何 Linux 机器上键入 `"man ascii"` 以查看您需要的字符的字节表示。
- 您注入的代码应该将寄存器 `%rdi` 设置为该字符串的地址。
- 当调用函数 `hexmatch` 和 `strncmp` 时，它们会将数据压入堆栈，覆盖 `getbuf` 使用的缓冲区所占据的部分内存。因此，您需要小心放置您的 **cookie** 的字符串表示的位置。

5 第二部分：返回导向编程 (ROP)

对程序 `RTARGET` 执行代码注入攻击比对 `CTARGET` 困难得多，因为它使用了两种技术来阻止此类攻击：

- 它使用**随机化**，因此堆栈位置在每次运行中都不同。这使得无法确定您注入的代码将位于何处。
- 它将包含堆栈的内存部分标记为**不可执行**，因此即使您可以将程序计数器设置为注入代码的开头，程序也会因分段错误而失败。

幸运的是，聪明的人设计了通过执行现有代码而不是注入新代码来完成有用事情的策略。最常见的形式被称为**返回导向编程 (Return-Oriented Programming, ROP)** [1, 2]。

ROP 的策略是识别现有程序中由一个或多个指令后跟 `ret` 指令组成的字节序列。这样的片段被称为 **gadget**。图 2 说明了如何设置堆栈以执行 n 个 gadget 的序列。在此图中，堆栈包含一系列 gadget 地址。每个 gadget 由一系列指令字节组成，最后一个字节是编码 `ret` 指令的 `0xc3`。当程序开始执行一个 `ret` 指令时，它将启动一个 gadget 执行链，每个 gadget 末尾的 `ret` 指令会导致程序跳转到下一个 gadget 的开头。

图 2：设置 gadget 序列以供执行。字节值 `0xc3` 编码 `ret` 指令。

一个 gadget 可以利用编译器生成的汇编语言语句对应的代码，尤其是在函数末尾的代码。在实践中，可能存在一些这种形式的有用 gadget，但不足以实现许多重要的操作。例如，一个编译后的函数在 `ret` 之前不太可能将 `popq %rdi` 作为其最后一条指令。幸运的是，对于像 x86-64 这样的面向字节的指令集，通常可以通过从指令字节序列的其他部分提取模式来找到 gadget。

例如，`rtarget` 的一个版本包含为以下 C 函数生成的代码：

```
1 void setval_210(unsigned *p)
2 {
3     *p = 3347663060U;
4 }
```

这个函数对于攻击系统似乎用处不大。但是，该函数的反汇编机器码显示了一个有趣的字节序列：


```
1 0000000000400f15 <setval_210>:  
2 400f15: c7 07 d4 48 89 c7 movl $0xc78948d4,(%rdi)  
3 400f1b: c3 retq
```

字节序列 `48 89 c7` 编码指令 `movq %rax, %rdi`。（请参阅图 3A 以获取有用的 `movq` 指令的编码。）该序列后跟字节值 `c3`，它编码 `ret` 指令。函数从地址 `0x400f15` 开始，序列从函数的第四个字节开始。因此，此代码包含一个起始地址为 `0x400f18` 的 gadget，它将 64 位值从寄存器 `%rax` 复制到寄存器 `%rdi`。

您的 `RTARGET` 代码包含许多类似于上面所示的 `setval_210` 函数的函数，位于我们称之为 **gadget farm** 的区域中。您的任务是识别 gadget farm 中有用的 gadget，并使用它们来执行类似于您在阶段 2 和 3 中所做的攻击。

重要事项： gadget farm 在您的 `rtarget` 副本中由函数 `start_farm` 和 `end_farm` 划定。不要尝试从程序代码的其他部分构造 gadget。

5.1 级别 2

对于阶段 4，您将重复阶段 2 的攻击，但在程序 `RTARGET` 上使用来自您的 gadget farm 的 gadget 来完成。您可以使用由以下指令类型组成的 gadget 来构造您的解决方案，并且只使用前八个 x86-64 寄存器（`%rax` 到 `%rdi`）。

- `movq`：这些编码如图 3A 所示。
- `popq`：这些编码如图 3B 所示。
- `ret`：此指令由单个字节 `0xc3` 编码。
- `nop`：此指令（发音为 "no op"，是 "no operation" 的缩写）由单个字节 `0x90` 编码。它的唯一作用是使程序计数器递增 1。

一些建议：

- 您需要的所有 gadget 都可以在 `rtarget` 代码中由函数 `start_farm` 和 `mid_farm` 划定的区域中找到。
- 您只需两个 gadget 即可完成此攻击。
- 当一个 gadget 使用 `popq` 指令时，它将从堆栈中弹出数据。因此，您的 exploit string 将包含 gadget 地址和数据的组合。

5.2 级别 3

在您开始阶段 5 之前，请停下来思考您迄今为止所完成的事情。在阶段 2 和 3 中，您使程序执行了您自己设计的机器码。如果 `CTARGET` 是一个网络服务器，您就可以将自己的代码注入到一台遥远的机器中。在阶段 4 中，您规避了现代系统用来阻止缓冲区溢出攻击的两个主要装置。尽管您没有注入自己的代码，但您能够注入一种通过拼接现有代码序列来运行的程序类型。您还获得了实验的 95/100 分。这是一个很好的分数。如果您有其他紧迫的义务，请考虑立即停止。

阶段 5 要求您对 `RTARGET` 进行 ROP 攻击，以调用函数 `touch3`，并传入指向您的 **cookie** 的字符串表示的指针。这看起来可能比使用 ROP 攻击调用 `touch2` 没有困难多少，但我们已经使其变得如此。此外，阶段 5 仅计 5 分，这并不能真正衡量它所需的努力。将其视为一个**额外加分题**，适用于那些想要超越课程正常期望的人。

图 3：指令的字节编码。所有值均以十六进制显示。

要解决阶段 5，您可以使用 `rtarget` 代码中由函数 `start_farm` 和 `end_farm` 划定的区域中的 gadget。除了阶段 4 中使用的 gadget 外，这个扩展的 farm 还包括不同 `movl` 指令的编码，如图 3C 所示。该 farm 的这部分字节序列还包含 2 字节指令，它们充当**功能性 NOP**，即它们不会改变任何寄存器或内存值。这些包括如图 3D 所示的指令，例如 `andb %a1, %a1`，它们对某些寄存器的低 4 字节进行操作，但不会改变它们的值。

一些建议:

- 您需要回顾 `movl` 指令对寄存器高 4 字节的影响, 如教科书第 183 页所述。
- 官方解决方案需要八个 gadget (并非所有都唯一)。

祝您好运, 玩得开心!

附录 A 使用 HEX2RAW

`HEX2RAW` 接受十六进制格式的字符串作为输入。在这种格式中, 每个字节值都由两个十六进制数字表示。例如, 字符串 `"012345"` 可以十六进制格式输入为 `"30 31 32 33 34 35 00"`。(回想一下, 十进制数字 x 的 ASCII 码是 `0x3x`, 并且字符串的结尾由一个空字节指示。)

您传递给 `HEX2RAW` 的十六进制字符应该用空白字符(空格或换行符)分隔。我们建议您在处理 exploit string 时使用换行符分隔其不同部分。`HEX2RAW` 支持 C 风格的块注释, 因此您可以标记出 exploit string 的各个部分。例如:

```
1 | 48 c7 c1 f0 11 40 00 /* mov $0x4011f0,%rcx */
```

请务必在起始和结束注释字符串 (`"/**"`、`"*/"`) 周围留出空格, 以便注释被正确忽略。

如果您在文件 `exploit.txt` 中生成了一个十六进制格式的 exploit string, 您可以通过几种不同的方式将原始字符串应用于 `CTARGET` 或 `RTARGET`:

1. 您可以设置一系列管道来通过 `HEX2RAW` 传递字符串。

```
1 | unix> cat exploit.txt | ./hex2raw | ./ctarget
```

2. 您可以将原始字符串存储在一个文件中并使用 I/O 重定向:

```
1 | unix> ./hex2raw < exploit.txt > exploit-raw.txt
2 | unix> ./ctarget < exploit-raw.txt
```

在 GDB 中运行时也可以使用此方法:

```
1 | unix> gdb ctarget
2 | (gdb) run < exploit-raw.txt
```

3. 您可以将原始字符串存储在一个文件中, 并提供文件名作为命令行参数:

```
1 | unix> ./hex2raw < exploit.txt > exploit-raw.txt
2 | unix> ./ctarget -i exploit-raw.txt
```

在 GDB 中运行时也可以使用此方法。

附录 B 生成字节码

使用 GCC 作为汇编器, OBJDUMP 作为反汇编器, 可以方便地生成指令序列的字节码。例如, 假设您编写了一个包含以下汇编代码的文件 `example.s`:


```

1  # Example of hand-generated assembly code
2  pushq $0xabcdef  # Push value onto stack
3  addq $17,%rax    # Add 17 to %rax
4  movl %eax,%edx   # Copy lower 32 bits to %edx

```

代码可以包含指令和数据的混合。# 字符右侧的任何内容都是注释。

您现在可以汇编和反汇编此文件：

```

1  unix> gcc -c example.s
2  unix> objdump -d example.o > example.d

```

生成的 `example.d` 文件包含以下内容：

```

1  example.o:      file format elf64-x86-64
2
3  Disassembly of section .text:
4
5  0000000000000000 <.text>:
6      0:   68 ef cd ab 00      pushq  $0xabcdef
7      5:   48 83 c0 11      add    $0x11,%rax
8      9:   89 c2          mov    %eax,%edx

```

底部的行显示了从汇编语言指令生成的机器码。每行的左侧都有一个十六进制数字，指示指令的起始地址（从 0 开始），而 : 字符后的十六进制数字指示指令的字节码。因此，我们可以看到指令 `push $0xABCDEF` 的十六进制格式字节码是 `68 ef cd ab 00`。

从该文件中，您可以获取代码的字节序列：

`68 ef cd ab 00 48 83 c0 11 89 c2`

然后，该字符串可以通过 `HEX2RAW` 传递，以生成目标程序的输入字符串。

或者，您可以编辑 `example.d` 以省略无关值并包含 C 风格的注释以提高可读性，从而得到：

```

1  68 ef cd ab 00 /* pushq $0xabcdef */
2  48 83 c0 11 /* add $0x11,%rax */
3  89 c2 /* mov %eax,%edx */

```

这也是您可以传递给 `HEX2RAW`，然后发送给目标程序的有效输入。